PBJ-2 Practical Assignment

Lab 1: End-to-End Loan Approval Automation

Name: Sunandha Kumar
Register No.: RA2411003010157

1. Objective

Build an end-to-end loan approval workflow on Camunda 8 that combines BPMN orchestration, a DMN decision table, an external Java/Spring Boot job worker, a REST Connector to a credit-assessment API, process variables, a human review task in Tasklist, and monitoring and incident recovery in Operate. All applicant data is synthetic.

2. Environment and Tools

Component	Version / details
Camunda 8 Run (c8run)	8.8.32, local: Zeebe, Operate, Tasklist, Connectors runtime 8.8.15
Camunda Desktop Modeler	5.49.0, target platform Camunda 8.8
Java / Spring Boot	OpenJDK 21, Spring Boot 3.5.16, camunda-spring-boot-starter 8.8.32
IDE / build	IntelliJ IDEA, Maven
Mock credit API	Spring REST controller in the same app, port 8090
Operate / Tasklist	http://localhost:8080/operate and /tasklist (demo / demo)

3. Architecture

The solution has two runtime parts on one machine:

- **Camunda 8 Run** executes the process (Zeebe), evaluates the DMN, runs the REST Connector, serves the Tasklist form and records history for Operate.
- **Spring Boot application** (port 8090) hosts the `validate-applicant` job worker and the mock credit-bureau endpoint `GET /api/credit-score/{applicantId}`.

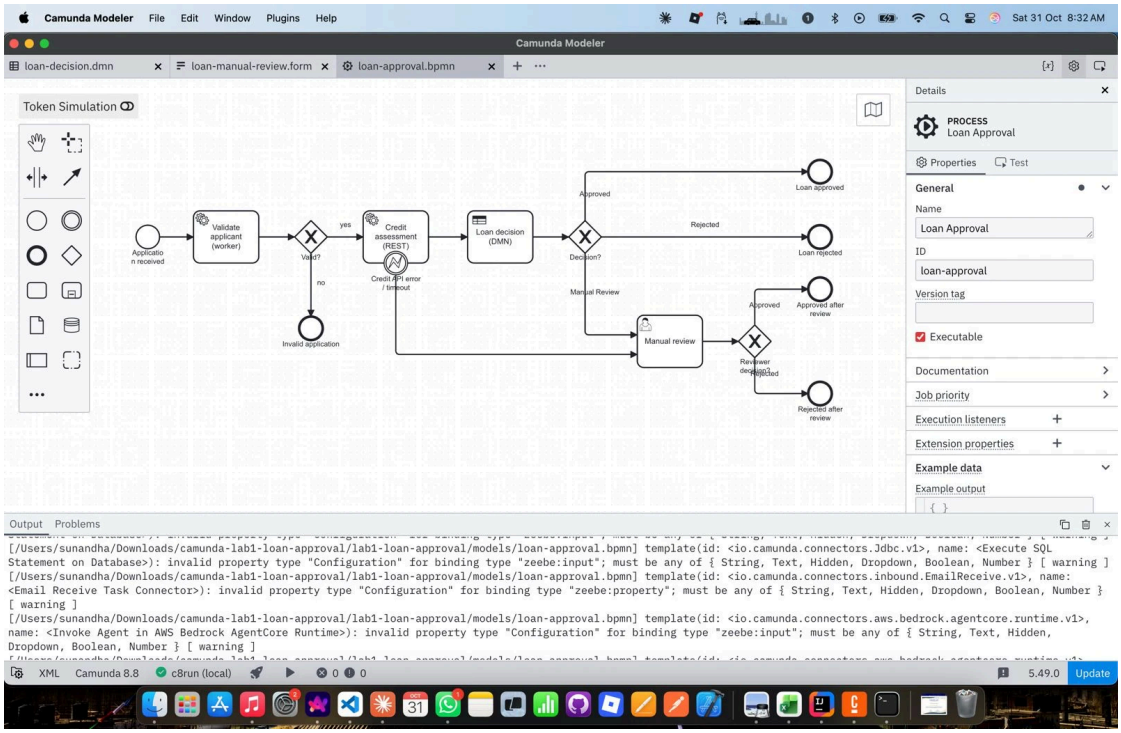


Figure 1: *Loan Approval BPMN process in Camunda Desktop Modeler (process id loan-approval)*

Execution flow: Application received → Validate applicant (Java worker) → Valid? gateway → Credit assessment (REST Connector) → Loan decision (DMN) → Decision? gateway → Loan approved / Loan rejected / Manual review. If the credit API returns an error or times out, an error boundary event routes the case straight to Manual review. After review, the Reviewer decision? gateway ends the instance at Approved after review or Rejected after review. Invalid applications end at Invalid application.

4. Implementation

4.1 BPMN process

Element	Type	Configuration
Validate applicant	Service task	Job type `validate-applicant`, 3 retries (Java worker)
Valid?	Exclusive gateway	`=valid = true` / `=valid = false`
Credit assessment	Service task (REST Connector)	Type `io.camunda:http-json:1`, GET mock API, 5 s read timeout
Credit API error / timeout	Error boundary event	Error code `CREDIT_API_ERROR`; output mapping exposes `creditApiError`
Loan decision	Business rule task	Calls DMN `loan-decision`, result variable `decision`
Decision?	Exclusive gateway	Routes on `decision` = Approved / Rejected / Manual Review
Manual review	User task (Camunda user task)	Assignee `demo`, linked form `loan-manual-review`
Reviewer decision?	Exclusive gateway	Routes on `reviewDecision` = Approved / Rejected

4.2 DMN decision table

Decision **`loan-decision`**, hit policy **FIRST** (rules are checked top to bottom; the first match wins). Inputs: **`creditScore`**, **`paymentToIncomeRatio`**, **`loanAmount`**. Output: **`decision`** (string).

Rule	creditScore	paymentToIncomeRatio	loanAmount	decision
R1	< 550	–	–	Rejected
R2	–	> 0.5	–	Rejected
R3	>= 700	<= 0.35	<= 50000	Approved
R4	–	–	–	Manual Review

These are simplified educational rules. **`paymentToIncomeRatio`** = (loanAmount / loanTermMonths) / monthlyIncome; interest is ignored. Sample evaluations from the test runs:

Applicant	creditScore	ratio	Rule hit	decision
A-100	780	0.09	R3	Approved
A-200	480	0.16	R1	Rejected
A-300	640	0.13	R4	Manual Review
A-600 (after fix)	760	0.10	R3	Approved

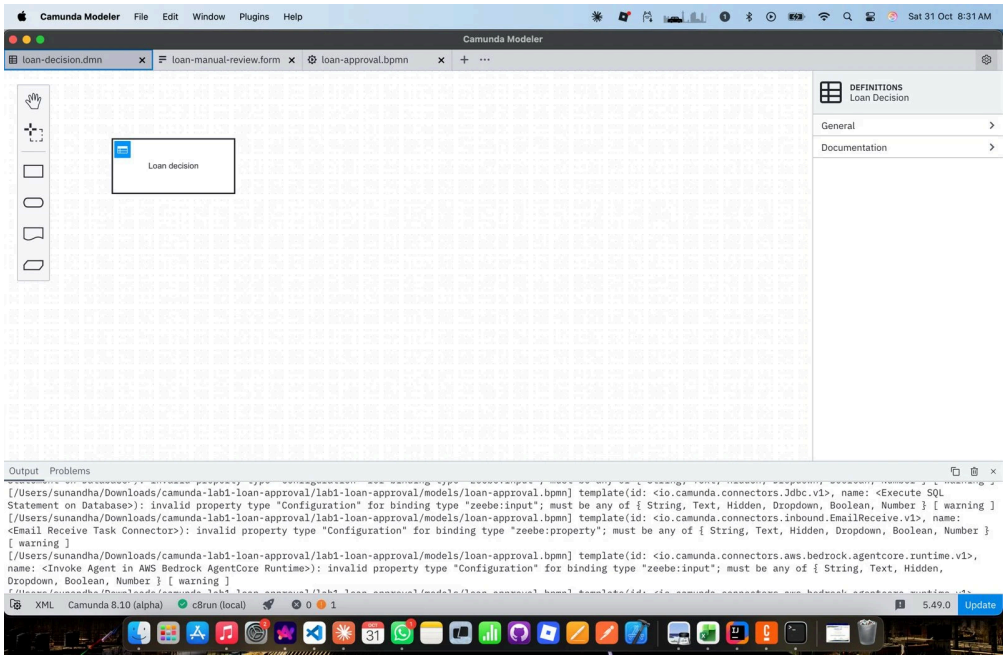


Figure 2: DMN decision requirements diagram (loan-decision) in Desktop Modeler

4.3 Spring Boot job worker

`ValidateApplicantWorker` is annotated with **`@JobWorker(type = "validate-applicant")`**. It reads the applicant variables, validates them, and returns new process variables (the job is auto-completed).

- **Business validation** (returns `valid = false` plus `validationErrors`): applicantId and applicantName required, age 18–75, loanAmount 1–500000, loanTermMonths 6–360, monthlyIncome > 0.
- **Derived values** when valid: `monthlyInstallment` and `paymentToIncomeRatio`.
- **Technical failure** (for the incident test): if `monthlyIncome` is missing, the worker throws `IllegalStateException`. The job fails, the 3 retries are used up and Zeebe raises a JOB_NO_RETRIES incident.

```
@JobWorker(type = "validate-applicant")
public Map<String, Object> validate(@Variable String applicantId, @Variable String applicantName,
    @Variable Integer age, @Variable Double loanAmount,
    @Variable Double monthlyIncome, @Variable Integer loanTermMonths) {
  if (monthlyIncome == null) throw new IllegalStateException("monthlyIncome is missing ...");
```

```
  ...
  return Map.of("valid", true, "monthlyInstallment", installment,
                "paymentToIncomeRatio", round(installment / monthlyIncome), ...);
}
```

Configuration (`application.properties`): `server.port=8090`, `camunda.client.grpc-address=http://localhost:26500`, `camunda.client.rest-address=http://localhost:8080`. Three JUnit tests cover the approve, invalid and missing-income paths (`mvn test`: 3 passed).

4.4 REST Connector and mock credit API

Setting	Value
Method / auth	GET, no authentication
URL	`="http://localhost:8090/api/credit-score/" + applicantId`
Timeouts	connection 20 s, read 5 s
Result expression	`={creditScore: response.body.creditScore, creditReference: response.body.reference}`
Error expression	`=if error != null then bpmnError("CREDIT_API_ERROR", error.message, {creditApiError: {code: error.code, message: error.message}}) else null`

Mock behaviour: A-100 → 780, A-200 → 480, A-300 → 640, A-600 → 760, any other ID → 650. IDs starting with **`ERR`** return HTTP 503; IDs starting with **`SLOW`** respond after 10 s, which exceeds the 5 s timeout.

```
GET http://localhost:8090/api/credit-score/A-100

200 {"applicantId":"A-100","creditScore":780,"reference":"CB-A-100"}


GET http://localhost:8090/api/credit-score/ERR-400

503 {"error":"Credit bureau unavailable"}
```

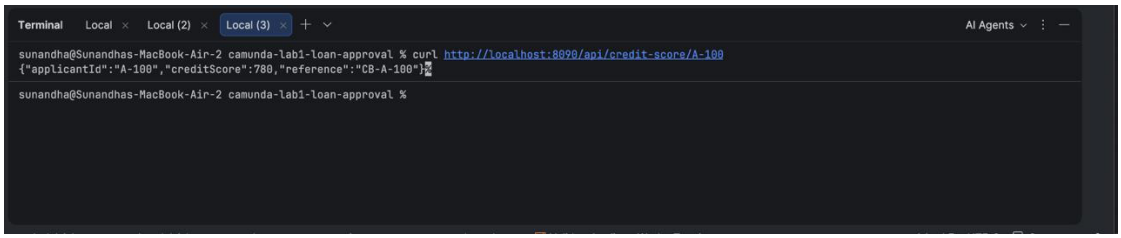


Figure 3: Mock credit-assessment API called from the terminal

4.5 Manual review task (Tasklist)

The Manual review user task uses the linked form **`loan-manual-review`**. It shows a summary of the application (applicant, amount, income, ratio, credit score, DMN decision and any credit API error) and collects **`reviewDecision`** (Approve / Reject, required) and an optional **`reviewComment`**.

4.6 Process variables

Variable	Type	Set by	Purpose
applicantId	String	Start	Synthetic reference; also used in the API URL
applicantName	String	Start	Display name
age	Number	Start	Eligibility check (18–75)
loanAmount	Number	Start	Requested amount; DMN input
monthlyIncome	Number	Start	Affordability; missing value triggers the incident test
loanTermMonths	Number	Start	Loan term (6–360)
valid	Boolean	Worker	Drives the Valid? gateway
validationErrors	List	Worker	Reasons an application is invalid
monthlyInstallment	Number	Worker	loanAmount / loanTermMonths
paymentToIncomeRatio	Number	Worker	DMN input
creditScore	Number	REST Connector	DMN input
creditReference	String	REST Connector	Credit bureau reference
creditApiError	String	Error boundary event	HTTP code / message shown to the reviewer
decision	String	DMN	Approved / Rejected / Manual Review
reviewDecision	String	Tasklist form	Reviewer's Approved / Rejected
reviewComment	String	Tasklist form	Reviewer's note

5. Deployment and Execution

The DMN, the form and the BPMN were deployed to the local cluster from Desktop Modeler (target c8run (local)). The Spring Boot app was started with **`mvn spring-boot:run`**, and one instance per scenario was started through the Orchestration Cluster REST API:

```
curl -u demo:demo -H "Content-Type: application/json" \

  -X POST http://localhost:8080/v2/process-instances -d @test-data/1-approved.json
```

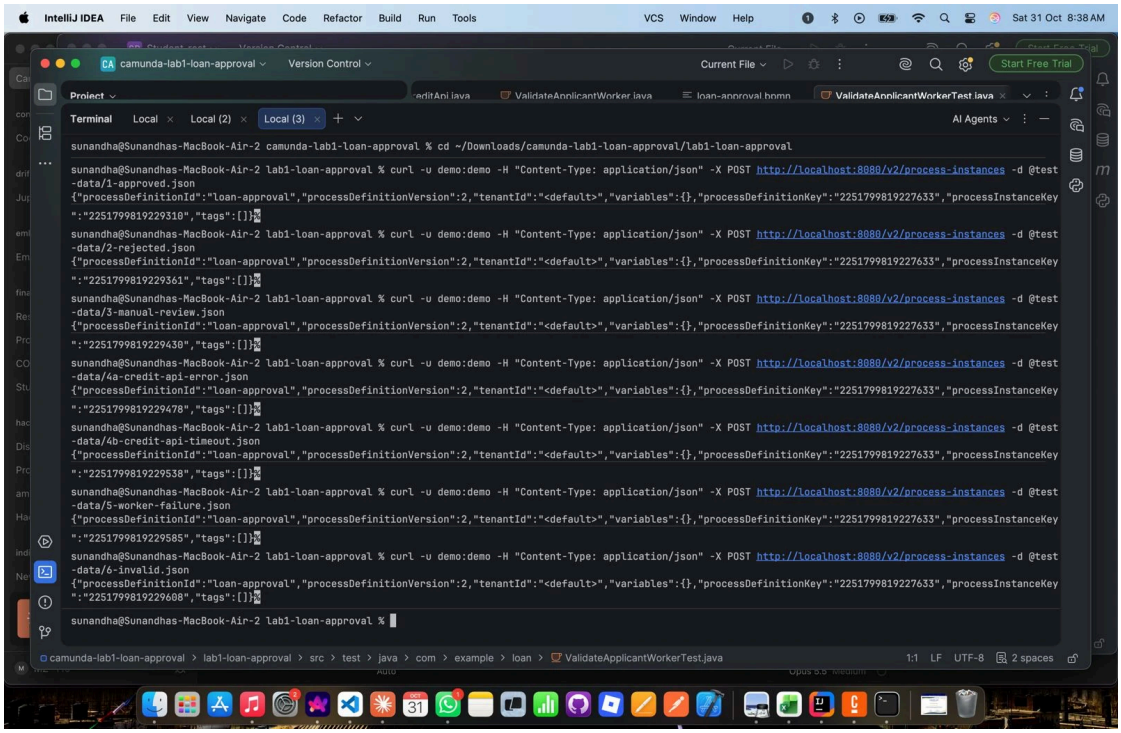


Figure 4: Starting the seven scenario instances (process version 2)

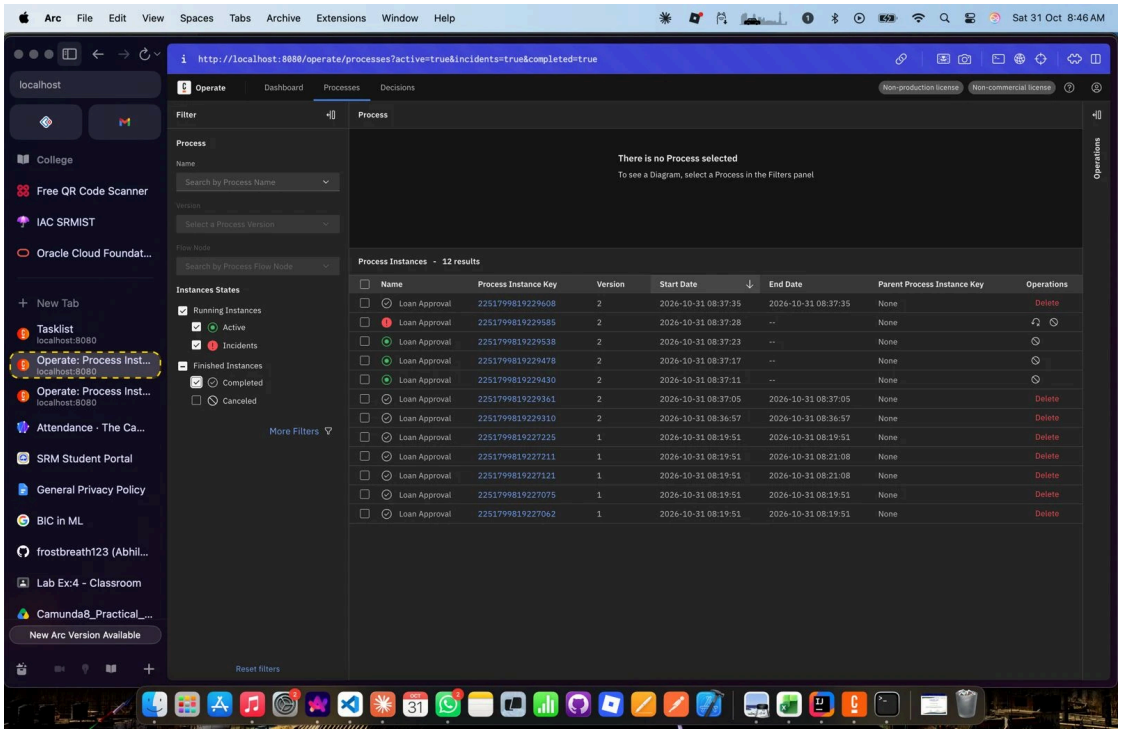


Figure 5: Operate: instance list with active, incident and completed instances

6. Test Results

All seven scenarios behaved as expected.

#	Scenario	Instance (ver.)	Actual outcome	Status
1	Meets approval criteria (A-100)	…229310 (v2)	Score 780, ratio 0.09, R3 → Loan approved	Pass
2	Fails eligibility (A-200)	…229361 (v2)	Score 480, R1 → Loan rejected	Pass
3	Needs manual review (A-300)	…233882 (v3)	Score 640, R4 → Tasklist → Approved after review	Pass
4a	Credit API error (ERR-400)	…234146 (v3)	HTTP 503 → boundary event → Tasklist → Rejected after review	Pass
4b	Credit API timeout (SLOW-500)	…233559 (v3)	408 timeout → boundary event → Tasklist → Approved after review	Pass
5	Worker failure + recovery (A-600)	…229585 (v2)	Incident → monthlyIncome added → Retry → Loan approved	Pass
6	Invalid input (A-700, age 16)	…229608 (v2)	valid = false → Invalid application	Pass

6.1 Scenario 1: Approved

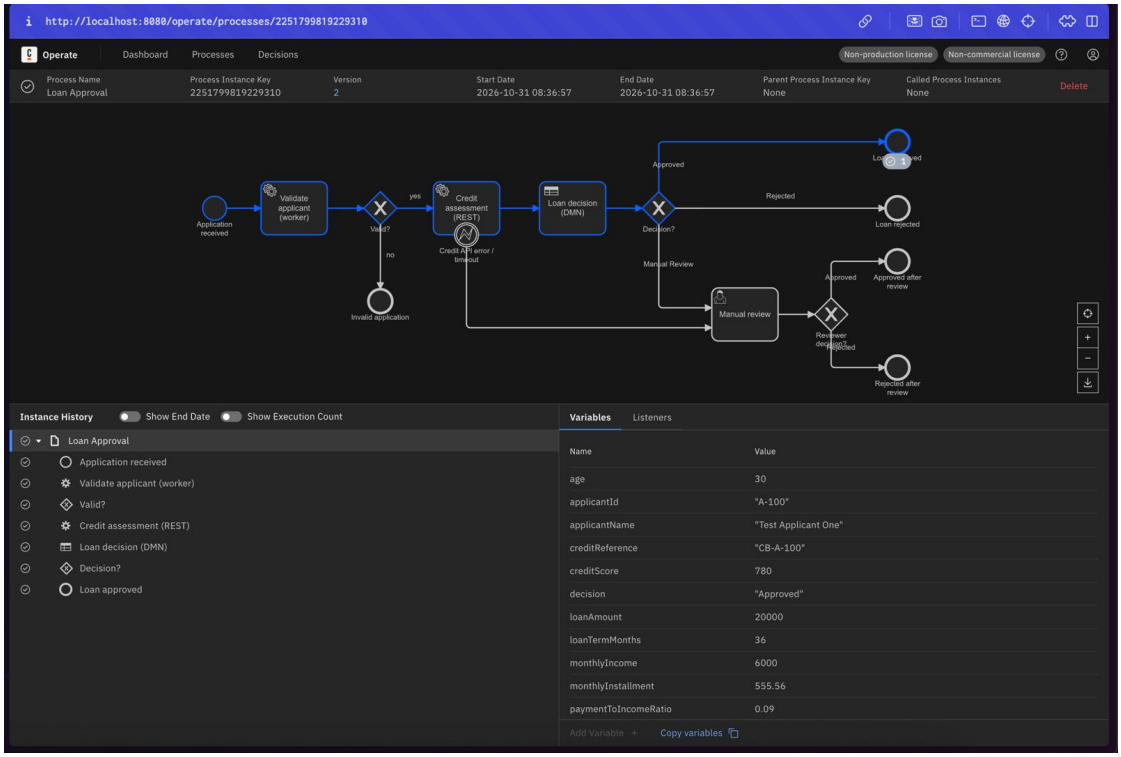


Figure 6: Scenario 1 (A-100): path to Loan approved, decision = "Approved"

6.2 Scenario 2: Rejected

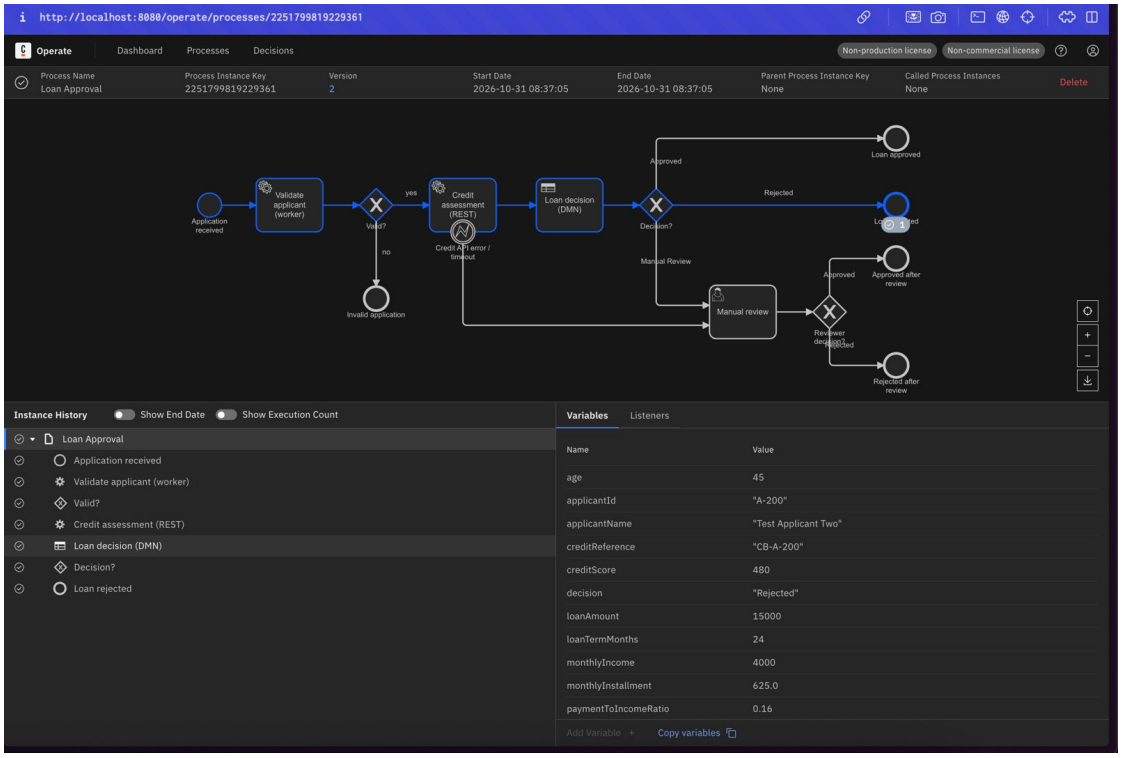


Figure 7: Scenario 2 (A-200): creditScore 480 → Loan rejected

6.3 Scenario 3: Manual review

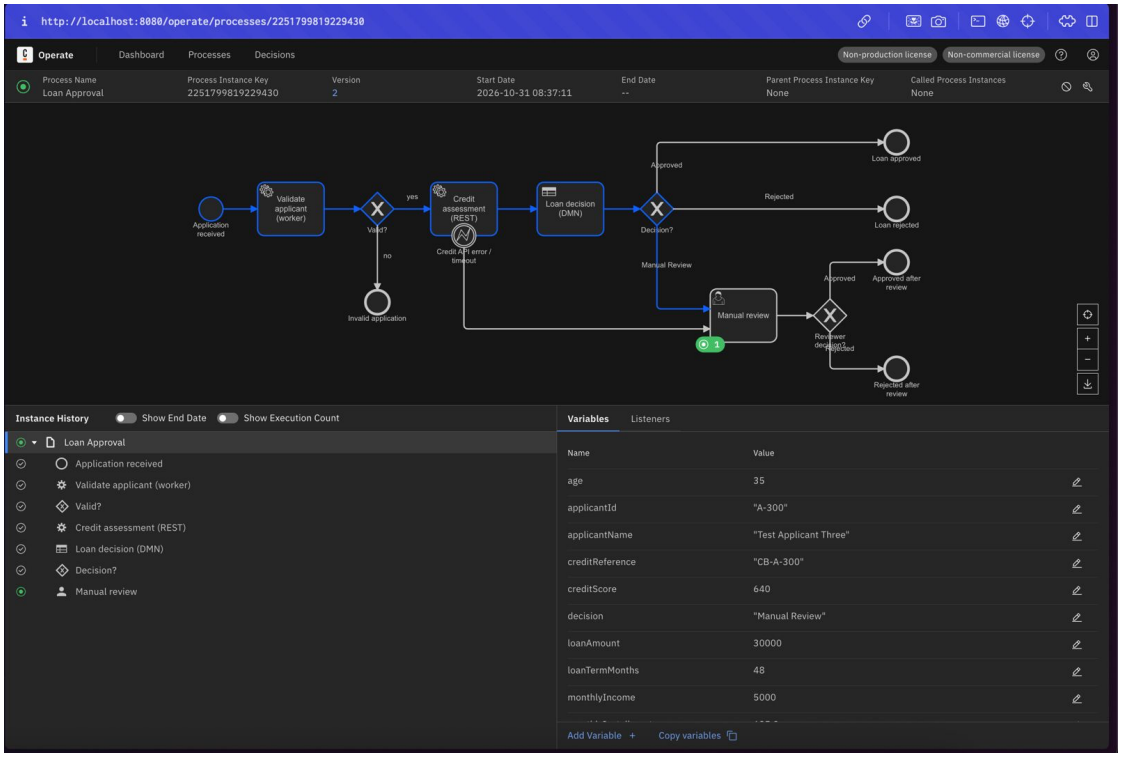


Figure 8: Scenario 3 (A-300): DMN returns Manual Review; instance waits at the user task

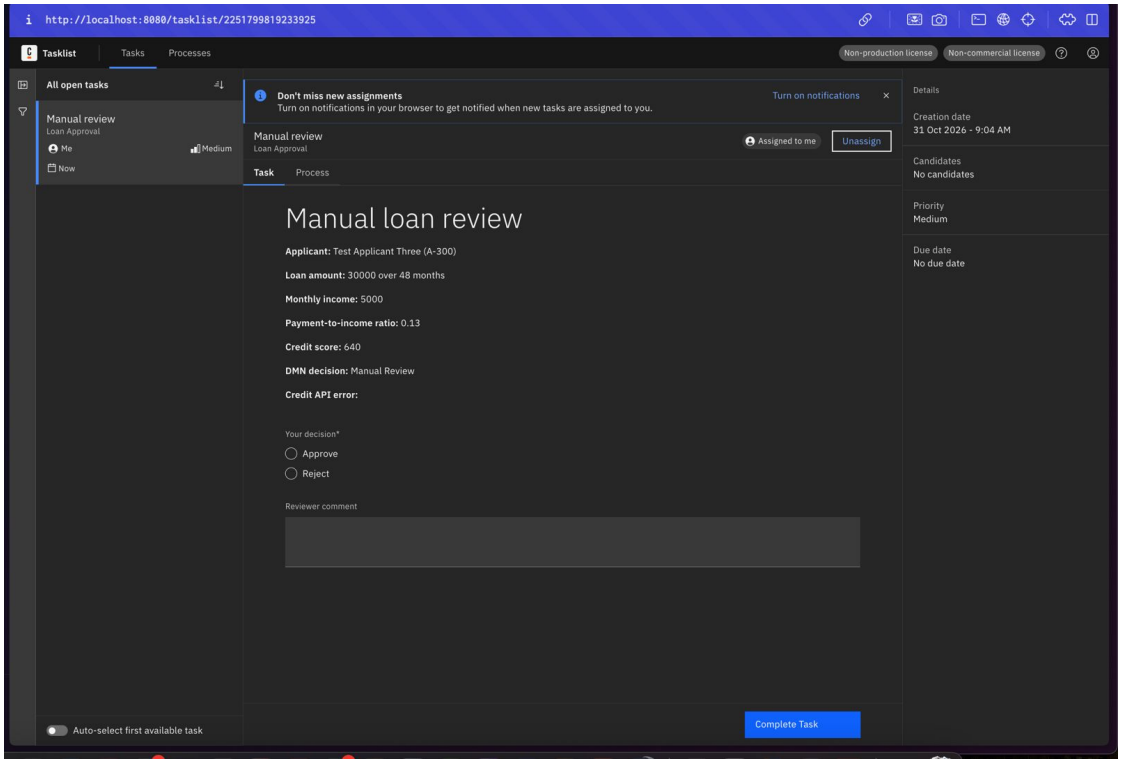


Figure 9: Scenario 3: review form in Tasklist (credit score 640, DMN decision Manual Review)

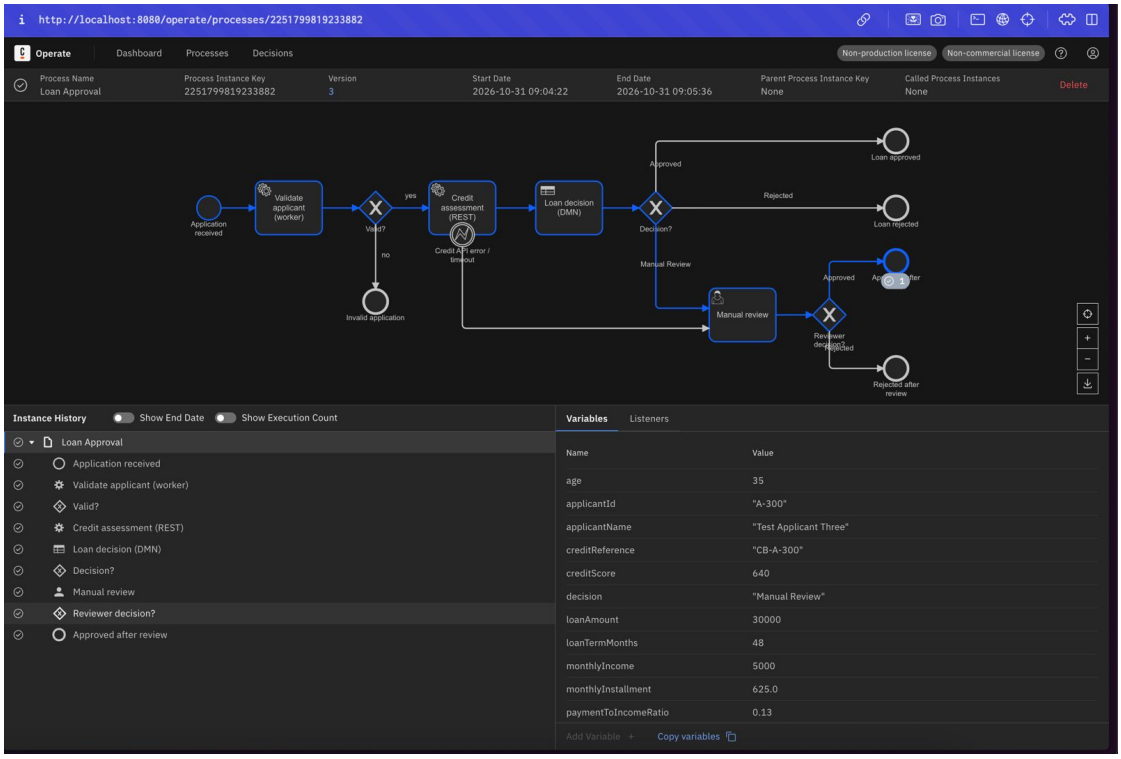


Figure 10: Scenario 3: reviewer approved → Approved after review

6.4 Scenario 4a: Credit API error

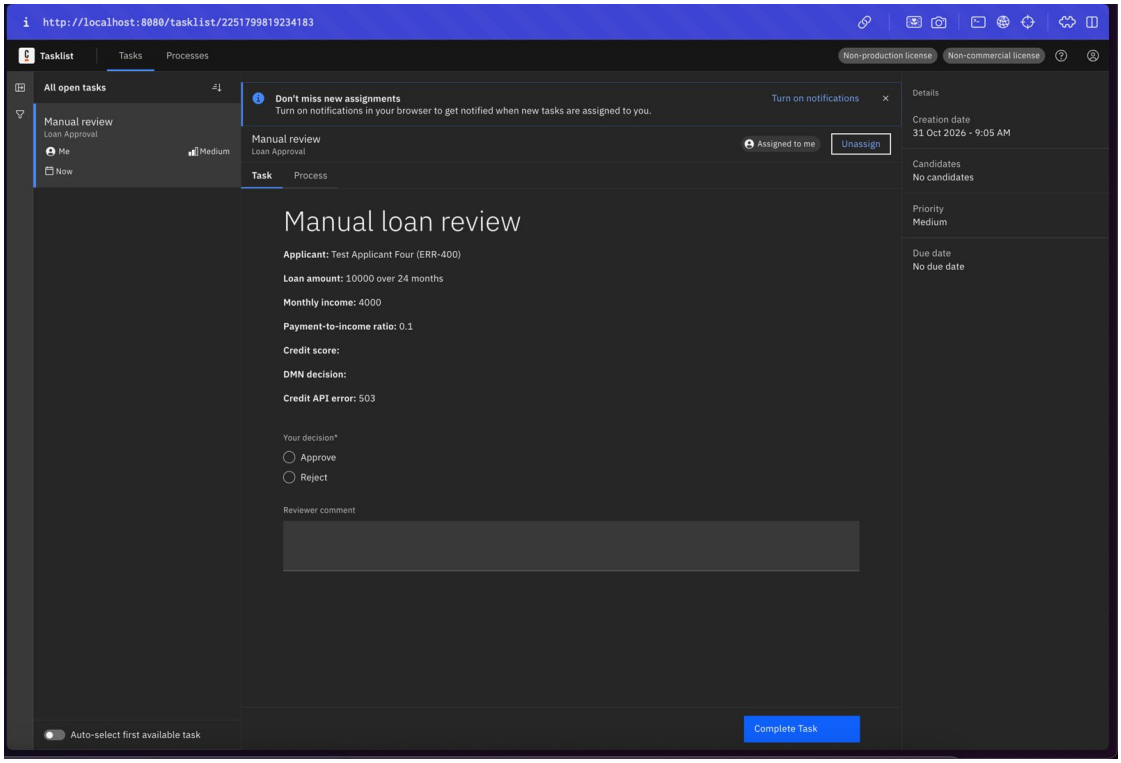


Figure 11: Scenario 4a (ERR-400): Tasklist shows Credit API error 503

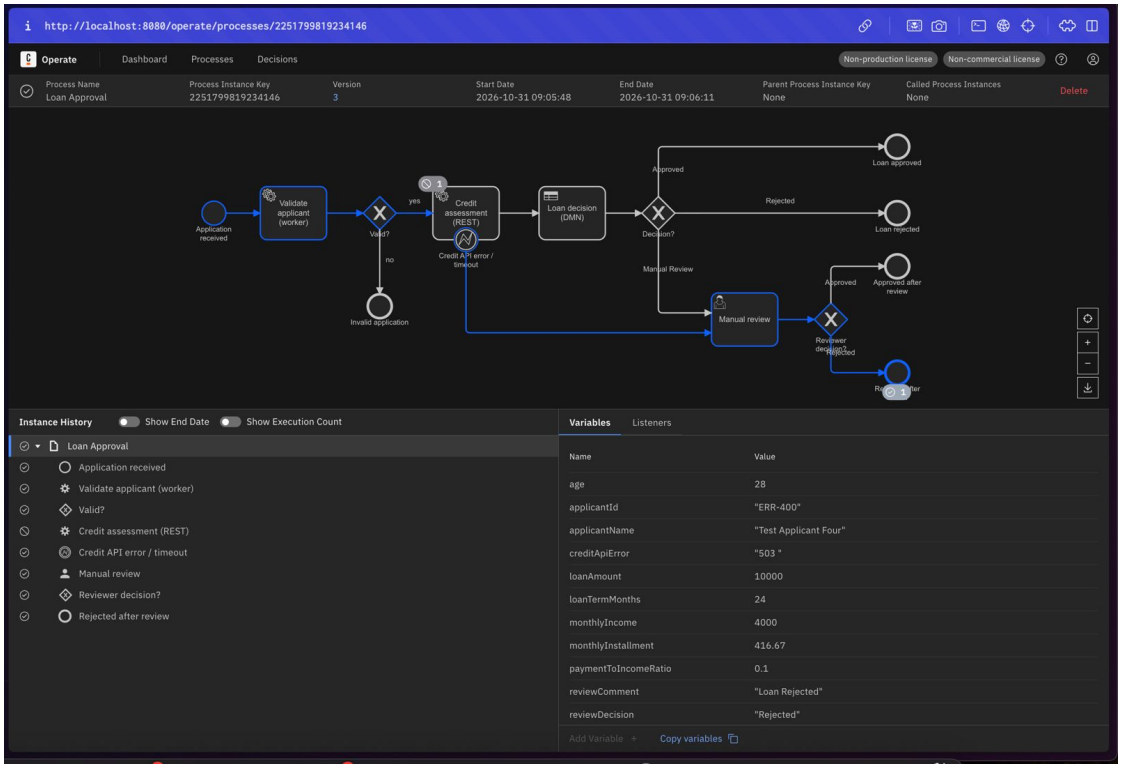


Figure 12: Scenario 4a: REST task interrupted by the error boundary event → Rejected after review

6.5 Scenario 4b: Credit API timeout

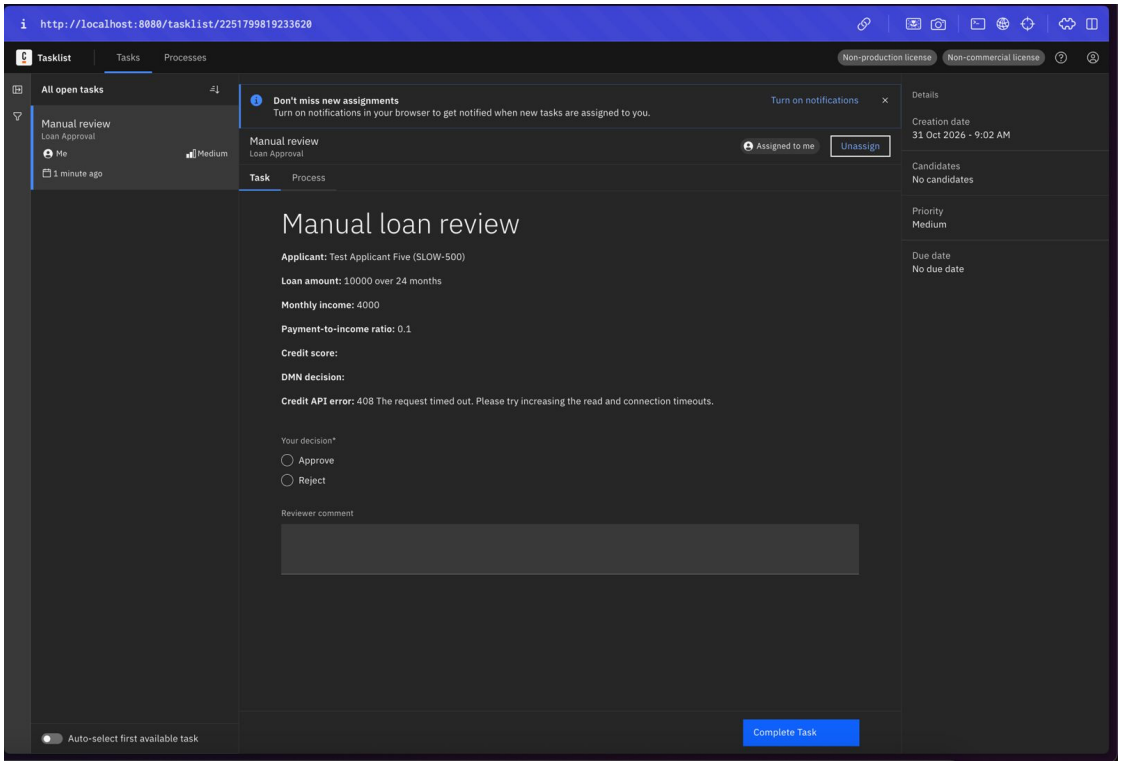


Figure 13: Scenario 4b (SLOW-500): Tasklist shows the 408 read timeout

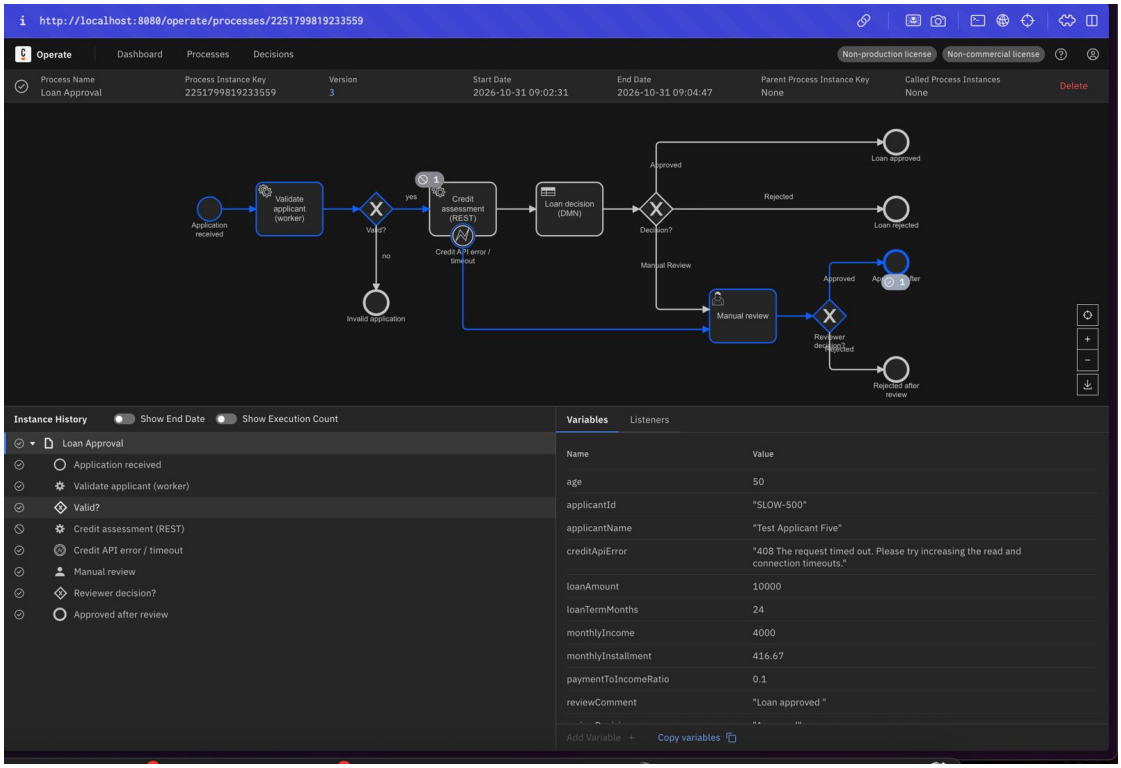


Figure 14: Scenario 4b: timeout routed to manual review → Approved after review

6.6 Scenario 6: Invalid application

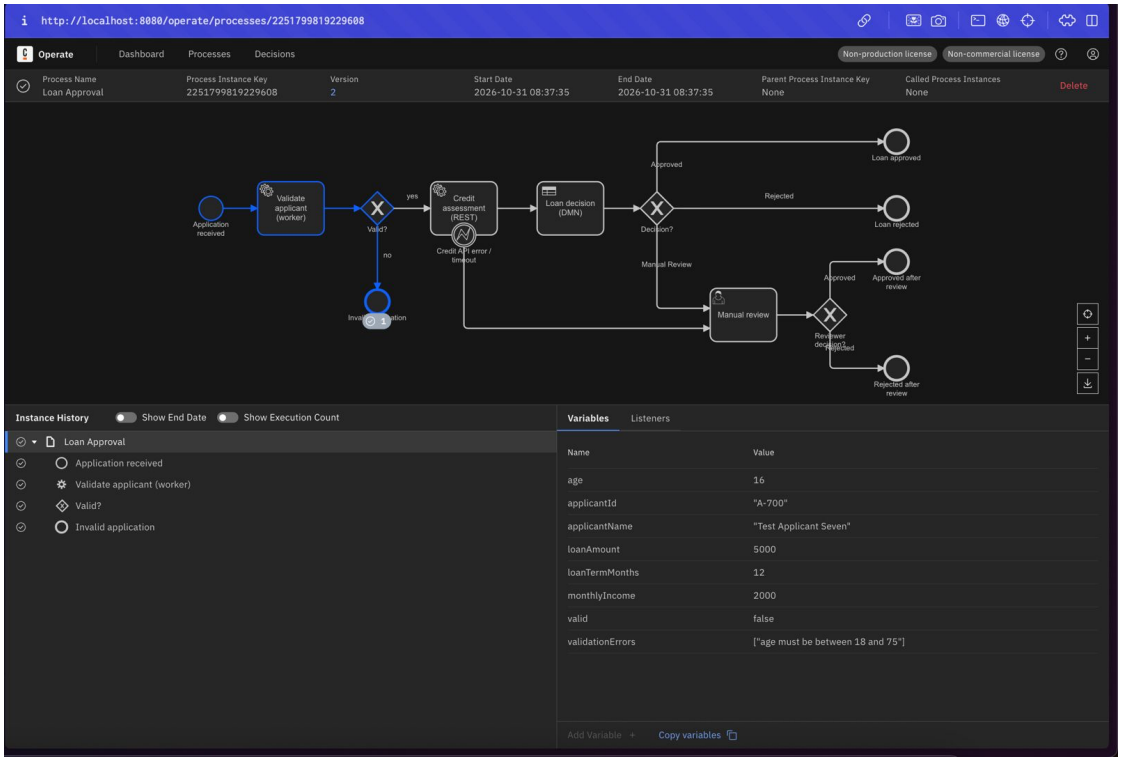


Figure 15: Scenario 6 (A-700): age 16 → valid = false → Invalid application

7. Incident Investigation and Recovery

Scenario 5 was started without **`monthlyIncome`** to simulate a worker failure.

1. **Detect:** Operate shows the instance with a red incident on Validate applicant (type JOB_NO_RETRIES) after the worker failed three times.
2. **Diagnose:** the incident message is `java.lang.IllegalStateException: monthlyIncome is missing for applicant A-600`. The Variables panel confirms `monthlyIncome` is absent.
3. **Correct:** `monthlyIncome = 5000` was added to the instance with Add Variable in Operate.
4. **Retry:** the incident was retried from Operate. The worker completed the job, the credit check returned 760, the DMN approved, and the instance ended at Loan approved at 09:14:07.

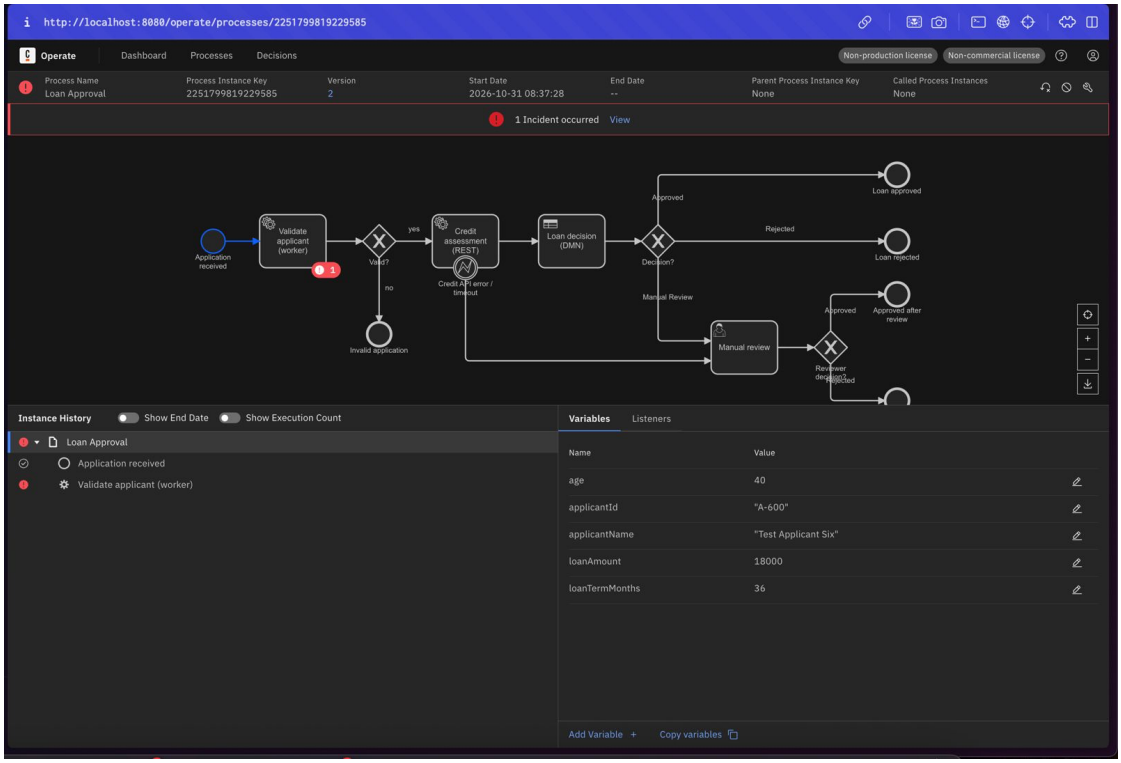


Figure 16: *Scenario 5 (A-600): incident on Validate applicant, monthlyIncome missing*

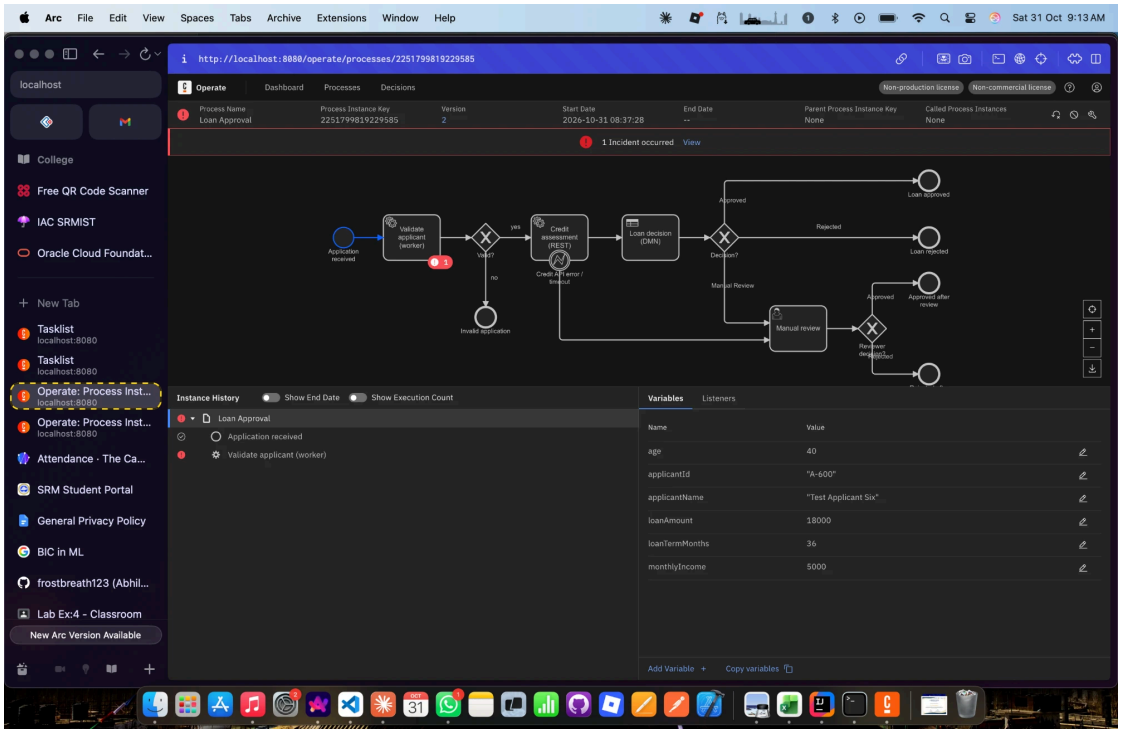


Figure 17: *Scenario 5: corrective action, monthlyIncome = 5000 added in Operate*

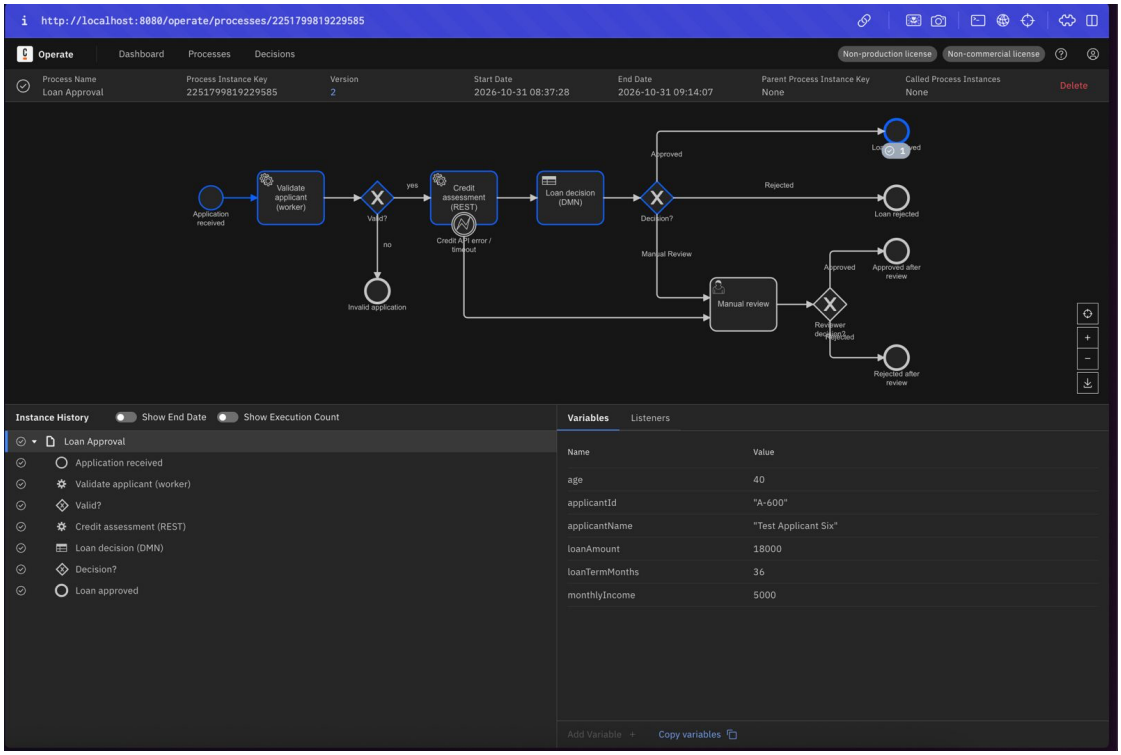


Figure 18: *Scenario 5: after Retry the instance completes at Loan approved*

7.1 Issue found during testing

In the first runs of scenarios 4a/4b (process version 2), the Tasklist form showed an empty Credit API error field although Operate showed the value. Investigation showed that variables passed with **`bpmnError(...)`** in a connector error expression are created **locally on the error boundary event** and are not visible to the following user task. The fix (version 3) was an output mapping on the boundary event: **`creditApiError = string join([creditApiError.code, creditApiError.message], " ")`**. After redeployment, the form showed 503 and 408 as expected, and scenarios 3, 4a and 4b were re-run on version 3.

8. Limitations

- Decision rules and the credit scoring are simplified for education; interest is ignored in the installment.
- The credit bureau is a deterministic mock without authentication; there is no real external service.
- Manual review is assigned to the single `demo` user; there are no candidate groups or SLAs.
- Everything runs on a local single-node c8run; no security, scaling or high-availability setup.
- Only the worker failure was recovered through an incident. Credit API errors are handled in the model by routing to a human instead.

9. Conclusion

The loan approval workflow was modelled, deployed and executed end to end on Camunda 8. BPMN orchestrated a Java job worker, a REST Connector and a DMN decision. Gateways routed each case by process variables, and a Tasklist form let a reviewer decide unclear and failed-credit-check cases. Operate was used to monitor every instance and to diagnose and recover a deliberate worker failure. All seven test scenarios passed.

Appendix: How to Run

1. Start Camunda: `cd c8run-8.8.32 && ./start.sh`
2. Start the worker and mock API: `cd lab1-loan-approval && mvn spring-boot:run`
3. Deploy `models/loan-decision.dmn`, `models/loan-manual-review.form` and `models/loan-approval.bpmn` from Desktop Modeler.
4. Start scenarios with the curl command in section 5, using the files in `test-data/`.
5. Complete review tasks in Tasklist and inspect instances in Operate.
6. Stop: Ctrl+C in the app terminal, then `./shutdown.sh`.